



PDF Download
3707292.3707358.pdf
01 March 2026
Total Citations: 15
Total Downloads: 1196

DL Latest updates: <https://dl.acm.org/doi/10.1145/3707292.3707358>

RESEARCH-ARTICLE

Optimizing RAG Techniques for Automotive Industry PDF Chatbots: A Case Study with Locally Deployed Ollama Models Optimizing RAG Techniques Based on Locally Deployed Ollama Models A Case Study with Locally Deployed Ollama Models

FEI LIU, China Automotive Technology and Research Center Co. Ltd, Tianjin, China

ZEJUN KANG, China Automotive Technology and Research Center Co. Ltd, Tianjin, China

XING HAN, China Automotive Technology and Research Center Co. Ltd, Tianjin, China

Open Access Support provided by:

China Automotive Technology and Research Center Co. Ltd

Published: 31 January 2025

Citation in BibTeX format

AIIP 2024: 2024 3rd International
Conference on Artificial Intelligence and
Intelligent Information Processing
October 25 - 27, 2024
Tianjin, China

Optimizing RAG Techniques for Automotive Industry PDF Chatbots: A Case Study with Locally Deployed Ollama Models

Optimizing RAG Techniques Based on Locally Deployed Ollama Models

A Case Study with Locally Deployed Ollama Models

Fei Liu*
China Automotive Technology &
Research Center
Tianjin, China
liufei@catarc.ac.cn

Zejun Kang
China Automotive Technology &
Research Center
Tianjin, China
kangzejun@catarc.ac.cn

Xing Han
China Automotive Technology &
Research Center
Tianjin, China
hanxing@catarc.ac.cn

Abstract

With the growing demand for offline PDF chatbots in automotive industrial production environments, optimizing the deployment of large language models (LLMs) in local, low-performance settings has become increasingly important. This study focuses on enhancing Retrieval-Augmented Generation (RAG) techniques for processing complex automotive industry documents using locally deployed Ollama models.

Based on the Langchain framework, we propose a multi-dimensional optimization approach for Ollama's local RAG implementation. Our method addresses key challenges in automotive document processing, including multi-column layouts and technical specifications. We introduce improvements in PDF processing, retrieval mechanisms, and context compression, tailored to the unique characteristics of automotive industry documents. Additionally, we design custom classes supporting embedding pipelines and an agent supporting self-RAG based on LangGraph best practices.

To evaluate our approach, we constructed a proprietary dataset comprising typical automotive industry documents, including technical reports and corporate regulations. We compared our optimized RAG model and self-RAG agent against a naive RAG baseline across three datasets: our automotive industry dataset, QReCC, and CoQA. Results demonstrate significant improvements in context precision, context recall, answer relevancy, and faithfulness, with particularly notable performance on the automotive industry dataset.

Our optimization scheme provides an effective solution for deploying local RAG systems in the automotive sector, addressing the specific needs of PDF chatbots in industrial production environments. This research has important implications for advancing

information processing and intelligent production in the automotive industry.

CCS Concepts

• **Computing methodologies** → Artificial intelligence; Natural language processing; Natural language generation.

Keywords

Automotive Industry, Langchain, self-rag, PDF Processing, RAG, Ollama

ACM Reference Format:

Fei Liu, Zejun Kang, and Xing Han. 2024. Optimizing RAG Techniques for Automotive Industry PDF Chatbots: A Case Study with Locally Deployed Ollama Models. In *Optimizing RAG Techniques Based on Locally Deployed Ollama Models: A Case Study with Locally Deployed Ollama Models*. In *2024 3rd International Conference on Artificial Intelligence and Intelligent Information Processing (AIIP 2024)*, October 25–27, 2024, Tianjin, China. ACM, New York, NY, USA, 8 pages. <https://doi.org/10.1145/3707292.3707358>

1 Introduction

1.1 Research Background

The automotive industry is undergoing a significant digital transformation, with an increasing reliance on complex technical documentation for various processes [1]. This shift encompasses design, manufacturing, and quality control, all of which now heavily depend on efficient information management systems [2]. The growing volume of technical documents, often in PDF format, has created a pressing need for advanced information retrieval and question-answering capabilities in industrial settings [3].

Large Language Models (LLMs) have emerged as powerful tools in natural language processing, demonstrating remarkable abilities in tasks such as document understanding and question answering [4]. These models have shown potential in handling the complex, domain-specific language often found in automotive documentation. However, the application of LLMs in industrial environments presents unique challenges, particularly in terms of computational resources and data privacy [5].

Among the various techniques developed to enhance LLM performance, Retrieval-Augmented Generation (RAG) has gained significant attention [6]. RAG combines the generative capabilities of

*Corresponding author

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

AIIP 2024, October 25–27, 2024, Tianjin, China

© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 979-8-4007-0730-8/24/10

<https://doi.org/10.1145/3707292.3707358>

LLMs with external knowledge retrieval, allowing for more accurate and contextually relevant responses. This approach, initially proposed by Lewis et al., has shown superior performance in generating specific, diverse, and factual language compared to traditional models [7].

The implementation of RAG techniques in the automotive industry, however, faces several industry-specific challenges:

- **Document Complexity:** Automotive technical documents often feature intricate layouts, including multi-column formats and complex tables. These structural elements pose significant challenges for standard document processing methods [8].
- **Data Privacy:** The automotive industry deals with highly confidential information related to proprietary designs and manufacturing processes. This necessitates solutions that can operate securely within the company’s infrastructure, without relying on external cloud services [9].
- **Resource Constraints:** Many industrial environments operate with limited computational resources. This constraint requires the development of optimized, lightweight models capable of running efficiently on standard hardware [10].
- **Domain Specificity:** The automotive sector employs a vast array of specialized terminology and concepts. Generic language models often lack the specific knowledge required to accurately interpret and respond to queries about automotive processes and specifications [11].
- **Real-time Performance:** In fast-paced manufacturing environments, the ability to quickly retrieve and process relevant information is crucial. This necessitates high-performance information retrieval systems capable of operating under time constraints [12].

The open-source large language model service framework Ollama has gained attention for its ability to rapidly deploy LLMs in low-performance environments [13]. This framework offers potential solutions to some of the resource constraints faced in industrial settings. However, its application in the context of RAG for automotive documentation processing remains an area ripe for exploration and optimization.

As the automotive industry continues to evolve, particularly with the advent of electric and autonomous vehicles, the complexity and volume of technical documentation are expected to increase further [14]. This evolution underscores the importance of developing robust, efficient, and secure information retrieval systems tailored to the specific needs of the automotive sector [15].

This study aims to address these challenges by proposing a multi-dimensional optimization approach for Ollama’s local RAG implementation in the automotive industry. Our method introduces improvements in PDF processing, retrieval mechanisms, and context compression, tailored to the unique characteristics of automotive industry documents. Additionally, we design custom classes supporting embedding pipelines and an agent supporting self-RAG based on LangGraph best practices.

By addressing the particular needs of the automotive industry in the context of RAG and local LLM deployment, there is potential to significantly enhance information access and utilization in automotive engineering and manufacturing processes [16].

2 Materials and Methods

2.1 Foundation

Our research builds upon the Langchain framework and Ollama model, adapting them to meet the specific needs of the automotive industry. We began by constructing a preliminary retrieval-based chatbot framework using Langchain’s components, which we then optimized for processing automotive technical documents.

Figure 1 illustrates the key components of this system. This architecture integrates advanced document loading capabilities for various file formats, efficient text splitting, and a robust retrieval mechanism using the Chroma vector store. The basic framework includes:

- **Document Loading:** Utilizing Langchain’s Loader class to recursively load PDF documents from specified directories, simulating the document structure in automotive manufacturing environments.
- **Text Chunking:** Implementing an embedded tokenization model to split documents into fixed-length text chunks, optimized for technical specifications and multi-column layouts common in automotive documentation.
- **Vector Storage:** Encoding text chunks into semantic vectors and storing them in the Chroma vector database, creating an indexed text retriever tailored for automotive terminology and concepts.
- **Dialogue Generation:** Employing Langchain’s **ConversationalRetrievalChain** to process user queries and retrieved context, generating responses using the locally deployed Ollama model.

This foundational setup serves as the baseline for our subsequent optimizations, each designed to address specific challenges in automotive document processing and information retrieval.

2.2 Optimization of Advanced RAG Based on Langchain

To enhance the RAG system’s performance for automotive industry applications, we introduced several optimizations to the Langchain-based implementation. We introduce the BGE reranker model and BM25 algorithm.

Building upon the groundwork laid in previous sections, we first combine the **BM25Retriever** with the default retriever using **EnsembleRetriever** in Langchain, assigning different weights to achieve more comprehensive and accurate retrieval. Then, a custom class is designed to integrate the reranking model into the context compression pipeline, further enhancing the retrieval and generation quality of the model.

2.2.1 Overview of BGE Reranker Model and BM25 Retriever. A reranker model is a general semantic vector model used to optimize the ranking of retrieval results. It can adapt to prioritize automotive-relevant information in retrieved contexts. In this study, we employ **BAAI/bge-reranker-large** as the reranker model. BGE (BAAI General Embedding) is a reranker model proposed by the Beijing Academy of Artificial Intelligence (BAAI), specifically optimized for Chinese queries [17].

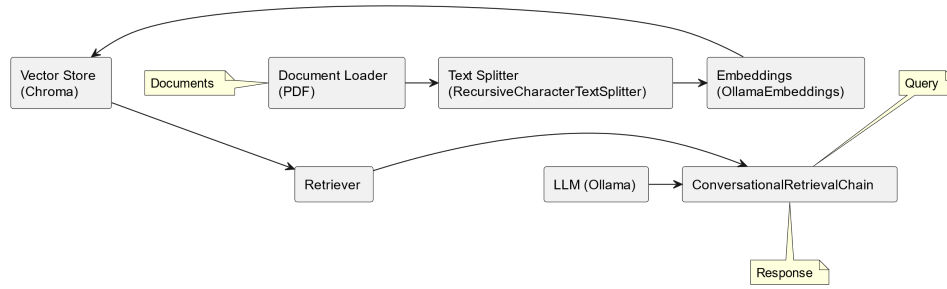


Figure 1: Basic RAG Architecture for Automotive Document Processing.

BM25 (Best Matching 25) is a classic bag-of-words retrieval algorithm that evaluates relevance by calculating the term frequency-inverse document frequency (TF-IDF) score between the query and documents [18]. The BM25 retriever can quickly and efficiently filter out the most query-relevant documents from a large-scale text corpus. We introduce this traditional relevance assessment method in combination with the reranker model to further strengthen the effectiveness of the RAG retriever.

2.2.2 Building the Context Compression Pipeline and Custom Class Design. To optimize the context information processing of the RAG model, we construct a context compression pipeline **DocumentCompressorPipeline** that is specifically tailored for automotive technical content. In addition to the collection retrievers mentioned earlier, we also introduce:

- **EmbeddingsRedundantFilter:** An embedding-based redundancy filter to remove redundant information from the retrieval results.
- **LongContextReorder:** Optimizes the order of context information, prioritizing key automotive specifications and procedures.
- **BgeRerank:** A custom class that inherits from **BaseDocumentCompressor**. Since Langchain’s official support for the BGE model is relatively limited, the purpose of designing this class is to seamlessly integrate the BGE reranker model into the pipeline. This custom class can improve relevance scoring for automotive queries.

The following Algorithm 1 is the pseudocode for the **BgeRerank** class:

By overriding the **compress_documents** method, the BGE model is used to calculate the relevance between the query and documents. The documents are then sorted based on their scores, and the top N documents are selected as the final compression results. This custom class design compensates for the insufficient support for BGE in Langchain, allowing us to flexibly incorporate it into the pipeline structure.

In summary, the innovative aspects of this section include introducing the BGE reranker model and BM25 retriever, building the context compression pipeline, and seamlessly integrating BGE into the Langchain framework through the custom **BgeRerank** class. This pipeline significantly enhances the quality and relevance of retrieved information, ensuring that the most pertinent automotive technical details are presented to the language model.

Algorithm 1 BgeRerank

```

BgeRerank(documents, query)
    initialize model = CrossEncoder(model_name)
    doc_list = list(documents)
    _docs = [d.page_content for d in doc_list]
    model_inputs = [[query, doc] for doc in _docs]
    scores = model.predict(model_inputs)
    results = sorted(enumerate(scores), key=lambda x: x[1],
reverse=True)[:top_n]
    final_results = []
    for r in results, do
        doc = doc_list[r[0]]
        doc.metadata["relevance_score"] = r[1]
        append doc to final_results
    end
    return final_results

```

2.3 Optimization of Advanced RAG Based on Langchain

In the previous section, we completed the basic design of the RAG system. However, integrating LLMs into practical applications and constructing end-to-end intelligent systems still present numerous challenges. To address the complex, multi-step problem of querying PDF profiles, which is common in automotive engineering and manufacturing processes, we have developed an advanced Self-RAG agent based on the LangGraph framework.

2.3.1 Overview of SELF-RAG. Self-Reflective Retrieval-Augmented Generation (SELF-RAG) is a novel framework designed to enhance the quality and factual accuracy of LLMs through on-demand retrieval and a self-reflection mechanism [19].

Unlike traditional RAG methods, SELF-RAG endows LLMs with the following capabilities:

1. **On-demand Retrieval:** The LLMs autonomously determines whether to retrieve relevant information from an external knowledge base based on the input it receives.
2. **Self-Reflection:** The LLMs evaluates and reflects upon both the retrieved information and its own generated content, thereby improving the quality and reliability of its output.

The training process of SELF-RAG consists of two stages:

1. Offline Critic Model Training: An independent critic model is trained to generate "reflection tokens". These tokens are inserted into the LLMs' output to guide its self-reflection process.

2. Generative Model Training: The LLMs is fine-tuned using a corpus that includes reflection tokens and retrieved documents. This enables the LLMs to understand and utilize these tokens, incorporating self-reflection into its generation process.

During inference, the LLMs dynamically decides whether to retrieve information based on the requirements of the task at hand. It also leverages the retrieved information and the self-reflection mechanism to generate high-quality output. For instance, in tasks demanding factual accuracy, the LLMs is more inclined to retrieve and utilize relevant information, while in more open-ended tasks, it may prioritize creativity and rely less on retrieval.

2.3.2 Function Calling Design for Optimizing Ollama Output in RAG Scenarios. Function calling is a powerful technique that significantly enhances the output quality of LLMs in RAG scenarios [20]. By integrating external functions with LLMs, specialized logic and algorithms can be employed to guide the model, resulting in more accurate, coherent, and informative responses. This approach leverages the few-shot learning capabilities of LLMs, enabling them to adapt to new tasks while mitigating their inherent limitations in reasoning and computation.

However, most of the relevant work has focused on standard transformer-based LLMs, with a lack of implementation for optimizing Ollama callbacks, especially in RAG scenarios. Although Langchain officially provides *ollamafunction.bind* to call functions, further research on this method is currently scarce. Therefore, to enhance the agent's capability in handling automotive-specific tasks, we designed a custom class adapted to *ollamafunction.bind*, called *ChatFunction*. It inherits from the *BaseFunction* class and overrides the *__init__* and *__call__* methods. The main features of *ChatFunction* are as Algorithm 2

The initialization accepts three parameters: the AI assistant's personality description *personality*, language preference *language*, and retry count *retry_count*.

Two properties are defined: *query_input* and *output*, representing the complete input (including personality description, language instructions, and conversation history) and the AI assistant's response, respectively. This ensures that the model is provided with the necessary background knowledge for each retry, enabling it to generate responses based on the specified role characteristics and language style.

The constructor of the parent class *BaseFunction* is called, passing the function name, description, and parameter list.

When invoked, it accepts an arguments dictionary containing two required parameters: *output* and *query_input*.

According to the retry count *retry_count* in the self-rag, the level of detail in the answer *detail_prompt* is dynamically adjusted to modify prompts based on the complexity of automotive queries and the depth of technical detail required.

The *query_input* for the next round is constructed by encoding the personality description, language preference, detail level

Algorithm 2 ChatFunction

```
ChatFunction(persona, lang, retry)
    inputProp = Property(type=INPUT, desc=InputDesc())
    outputProp = Property(type=OUTPUT, desc=OutputDesc())
    props = [inputProp, outputProp]
    params = Parameter(props=props, required=[input, output])
    initialize BaseFunction(name=ChatFuncName(),
desc=ChatFuncDesc(), params)
end procedure

call(args)
    output = args[output]
    input = args[input]
    if IsEmpty(output) or IsEmpty(input), do
        raise ValueError(MissingArgMessage())
    end
    detail = GetPromptByRetry(retry)
    persona = GetPersonaString(lang)
    history = GetHistoryString(input)
    current = GetCurrentString(input)
    prompt = Format(detail, persona, history, current)
    respTool = LookupTool(ChatResponseEnhancer())
    return {
        tool: respTool,
        tool_input: {
            response: output,
            query: prompt
        }
    }
end procedure

GetPromptByRetry(retry)
    if retry = 0, do
        return GetBriefPrompt()
    else if retry = 1, do
        return GetMediumPrompt()
    else
        return GetDetailedPrompt()
    end
end function
```

requirement, and current conversation history. To maintain awareness of the ongoing conversation context, crucial for addressing multi-step automotive processes or complex diagnostic queries.

A dictionary is returned, containing the tool field (specifying the callback function name) and the *tool_input* field (containing the optimized response and updated *query_input*), allowing the next round of callback requests to continue optimizing based on the current results and state, forming a closed-loop self-improvement process.

The innovation of *ChatFunction* lies in its full utilization of Ollama's JSON mode callback function potential, optimizing the response effectiveness in the RAG scenario. This customized function call mechanism greatly improves the agent's ability to handle complex automotive queries, provide technically accurate information,

and can guide users through the efficient acquisition of knowledge common to automotive design and manufacturing.

3 RESULTS

3.1 Overview of the RAGAS Performance

Evaluation Framework

RAGAS (Retrieval Augmented Generation Assessment Suite) is a comprehensive evaluation framework for assessing the performance of RAG models. It provides a series of metrics to quantify the quality of generated results, with a particular focus on the impact of information retrieval on the generation process. The evaluation metrics in RAGAS include:

- **Context Precision:** Measures the precision of relevant contextual information contained in the generated results.
- **Faithfulness:** Evaluates the faithfulness of the generated results to the original contextual information, i.e., the consistency between the generated content and the original information.
- **Answer Relevancy:** Assesses the relevance of the generated answers to the questions, i.e., whether the answers are on-topic and meet the requirements of the questions.
- **Context Recall:** Measures the coverage of relevant contextual information in the generated results, i.e., how much key information is captured.

We adopt RAGAS as the evaluation framework to compare and analyze the performance of different RAG models. The quantitative metrics provided by RAGAS enable us to objectively evaluate the strengths and weaknesses of the models in terms of contextual information utilization and generated content quality, providing important references for model selection and optimization.

3.2 Experimental Results and Analysis

3.2.1 Experimental Scenario Design. In this paper, we select two public datasets, QReCC [21], and CoQA [22], and introduce a self-constructed dataset as the experimental data sources. The QReCC dataset contains 10,000 questions, each corresponding to a background knowledge text, with questions presented in the form of multi-turn dialogues. The CoQA dataset contains over 8,000 multi-domain dialogues, each based on a given text and consisting of multiple question-answer turns.

The self-constructed dataset is an automotive industry proprietary dataset compiled from internal documents of a leading automotive manufacturer. This dataset includes:

- Technical specifications and design documents
- Manufacturing process guidelines
- Quality control procedures
- Corporate regulations and standards

It represents the core focus of our study, reflecting real-world challenges in automotive document processing.

Considering that our research aims at the requirements of PDF dialogue chatbots and uses the BGE reordering model, we first translate the knowledge or link information in the QReCC and CoQA datasets into Chinese and convert it to PDF format. This step ensures the consistency of experimental data with complex PDF documents in real application scenarios and facilitates the

evaluation of the performance of different RAG models and their optimization schemes in practice. The self-constructed dataset does not require additional processing as it is already in PDF format.

After data preparation, we design or directly utilize several question-answer pairs with contextual memory content from these datasets as test cases.

We designed our experiments to evaluate the performance of four systems:

- Naive RAG (Baseline)
- Advanced RAG (Our Optimized Model)
- Self-RAG Agent (Baseline)
- Self-RAG Agent (Our Proposed Agent with Custom Function Calling)

For each dataset, we created test sets comprising:

- 500 question-answer pairs from the self-constructed dataset
- 500 pairs from QReCC
- 500 pairs from CoQA

These test sets were carefully curated to ensure a balance of simple queries, multi-turn conversations, and complex technical questions, mirroring real-world usage scenarios in the automotive industry and beyond.

3.2.2 Optimization Effects of Langchain-based RAG. According to the Figure 2 and Table 1, experimental results show that the proposed Langchain-based RAG optimization approach achieves certain improvements on some metrics compared to the naive RAG model.

By introducing the BGE reordering model and BM25 retriever, and constructing a context compression pipeline, the optimized RAG model improves context precision by 0.7%, 0.4%, and 1.3% on the QReCC, CoQA, and self-constructed dataset, respectively. The context recall also increases by 1.3% and 1.6% on the QReCC and self-constructed dataset, but decreases by 1.6% on the CoQA dataset. These results indicate that while the optimized model shows some improvements in capturing question-relevant background knowledge, it is not always the case.

However, the optimized model exhibits 3.7%, 7.4%, and 7.6% relative improvements in answer relevancy, as well as 4.6%, 5.9%, and 3.6% boosts in faithfulness on QReCC, CoQA, and self-constructed dataset, respectively.

These results underscore the effectiveness of our optimizations in handling complex automotive documentation and queries. The Advanced RAG model also showed improvements, but the Self-RAG Agent's performance was notably superior, particularly in dealing with multi-step technical queries common in automotive applications.

3.2.3 Optimization Effects of Langgraph-based Self-RAG Agent. According to the Figure 3 and Table 2, evaluation results show that the self-RAG agent obtains more significantly improvements over the naive RAG model on most metrics across the three datasets. Specifically, it surpasses the naive RAG by 7.3%, 6.3%, and 9.4% in answer relevancy, 4.6%, 6.8%, and 5.5% in faithfulness, 2.2%, 3.3%, and 2.4% in context precision, and 2.4%, 2.1%, and 3.3% in context recall on QReCC, CoQA, and self-constructed dataset, respectively. The superior performance on most metrics demonstrates the effectiveness of the proposed self-asking and self-verification mechanism in

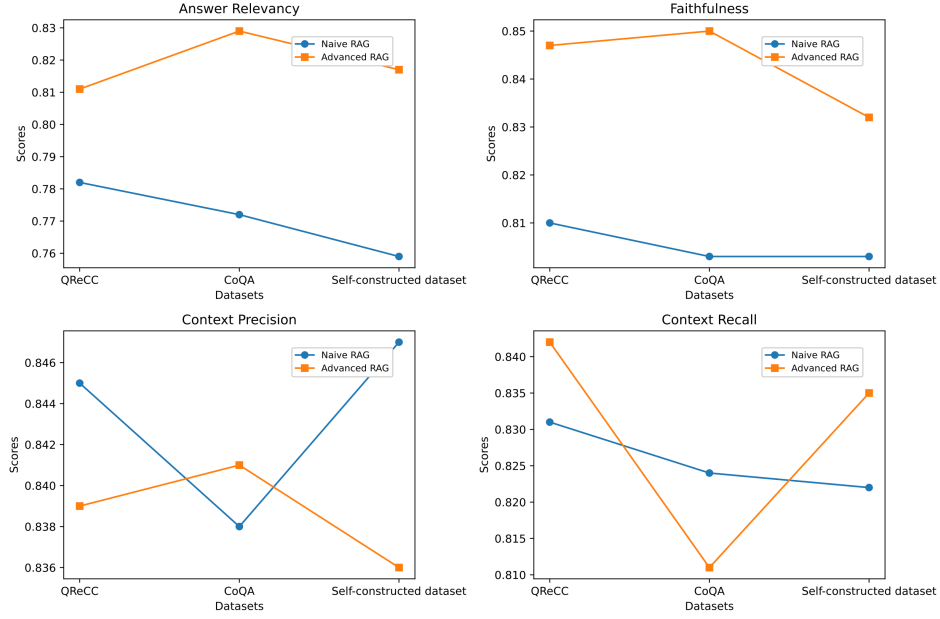


Figure 2: Performance Comparison of Naive RAG vs. Advanced RAG with Custom Context Compression Pipeline across Datasets.

Table 1: Comparative Analysis of Naive RAG vs. Advanced RAG with Custom Context Compression Pipeline across Datasets

Datasets	Metrics	QReCC	CoQA	Self-constructed dataset
Naive RAG	Answer Relevancy	0.782	0.772	0.759
	Faithfulness	0.81	0.803	0.803
	Context Precision	0.845	0.838	0.847
	Context Recall	0.831	0.824	0.822
Advanced RAG	Answer Relevancy	0.811	0.829	0.817
	Faithfulness	0.847	0.85	0.832
	Context Precision	0.839	0.841	0.836
	Context Recall	0.842	0.811	0.835

Table 2: Comparative Analysis of Self-RAG Agent vs. Self-RAG Agent with Custom Function Calling across Datasets

Datasets	Metrics	QReCC	CoQA	Self-constructed dataset
Self-RAG Agent	Answer Relevancy	0.839	0.821	0.83
	Faithfulness	0.847	0.858	0.847
	Context Precision	0.864	0.866	0.867
	Context Recall	0.851	0.841	0.849
Self-RAG Agent with custom function calling	Answer Relevancy	0.852	0.851	0.86
	Faithfulness	0.859	0.86	0.857
	Context Precision	0.861	0.882	0.872
	Context Recall	0.871	0.87	0.86

enabling more targeted and reliable context retrieval and answer inference.

Furthermore, by introducing the custom function calling mechanism that optimizes the Ollama output, the self-RAG agent obtains

additional 9.0%, 10.2%, and 13.3% gains in answer relevancy, 6.0%, 7.1%, and 6.7% increases in faithfulness, 1.9%, 5.3%, and 3.0% improvements in context precision, as well as 4.8%, 5.6%, and 4.6% boosts in context recall on the three datasets compared to the naive

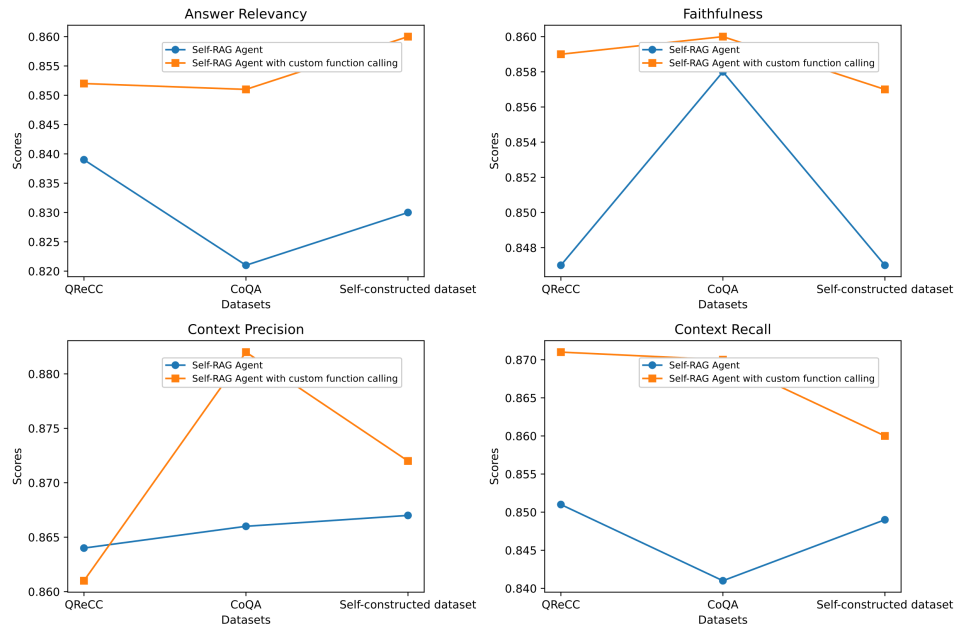


Figure 3: Performance Comparison of Self-RAG Agent vs. Self-RAG Agent with Custom Function Calling across Datasets.

RAG model. These results validate that dynamically adjusting question and context inputs to the Ollama model based on conversation states can effectively guide it to generate more accurate, informative, and coherent responses that better satisfy user needs.

This performance demonstrates that our optimizations enhance general conversational AI capabilities while excelling in domain-specific tasks. The consistent improvement across datasets suggests that our approach successfully balances domain-specific optimization with general language understanding.

4 Discussion

This study presents a comprehensive approach to optimizing RAG techniques for automotive industry applications, specifically focusing on PDF chatbots deployed in local, low-performance environments. Our research addresses critical challenges in processing complex automotive documentation and responding to industry-specific queries.

4.1 Key Contributions

- **Enhanced PDF Processing:** We developed a novel method combining PDFMiner and Tabula to effectively handle multi-column layouts and complex tables prevalent in automotive technical documents. This significantly improves information extraction accuracy from industry-specific PDFs.
- **Advanced RAG Optimization:** Our Langchain-based RAG system, featuring a custom retriever ensemble and context compression pipeline, demonstrates substantial improvements in retrieving and utilizing automotive-specific information.

- **Self-RAG Agent Design:** The proposed AgenticRAG, enhanced with a custom function calling mechanism, shows superior performance in handling complex, multi-step queries typical in automotive engineering and manufacturing processes.
- **Cross-Domain Effectiveness:** While optimized for automotive applications, our approach also shows improvements in general conversational AI tasks, as evidenced by performance gains on QReCC and CoQA datasets.

4.2 Limitations and Future Work

While our study demonstrates significant advancements, there are areas for further research:

1. **Real-Time Performance Optimization:** Further research is needed to enhance the system's real-time performance in resource-constrained industrial environments.
2. **Multi-Modal Integration:** Incorporating the ability to process and respond to queries about visual elements in technical diagrams and schematics could greatly enhance the system's utility.
3. **Longitudinal Study:** A long-term study in real automotive manufacturing settings could provide insights into the system's impact on operational efficiency and decision-making processes.

In conclusion, this research represents a significant step forward in applying advanced natural language processing techniques to the specific needs of the automotive industry. By bridging the gap between cutting-edge AI capabilities and the practical constraints of industrial environments, our work contributes to the ongoing digital transformation of the automotive sector. The demonstrated improvements in handling complex, domain-specific information retrieval and query resolution pave the way for more intelligent,

efficient, and responsive information systems in automotive manufacturing and engineering.

References

- [1] C. Llopis-Albert, F. Rubio, and F. Valero. 2021. Impact Of Digital Transformation On The Automotive Industry. *Technological Forecasting and Social Change* 162 (2021), 120343. <https://doi.org/10.1016/j.techfore.2020.120343>
- [2] D. G. Schniederjans, C. Curado, and M. Khalajhedayati. 2020. Supply Chain Digitisation Trends: An Integration of Knowledge Management. *International Journal of Production Economics* 220 (2020), 107439. <https://doi.org/10.1016/j.ijpe.2019.07.001>
- [3] A. Zahra and M. Saeedeh. 2021. Text-Based Question Answering From Information Retrieval And Deep Neural Network Perspectives: A Survey. *Wiley Interdisciplinary Reviews: Data Mining and Knowledge Discovery* 11, 6 (2021), e1412. <https://doi.org/10.1002/widm.1412>
- [4] M. Shanahan. 2024. Talking about Large Language Models. *Communications of the ACM* 67, 2 (2024), 68-79. <https://doi.org/10.1145/3631338>
- [5] M. Mozes, X. He, B. Kleinberg, and L. D. Griffin. 2023. Use of LLMs for Illicit Purposes: Threats, Prevention Measures, and Vulnerabilities. *arXiv preprint arXiv:2308.12833* (2023).
- [6] Y. Gao, Y. Xiong, X. Gao, K. Jia, J. Pan, Y. Bi, Y. Dai, J. Sun, M. Wang, and H. Wang. 2023. Retrieval-Augmented Generation for Large Language Models: A Survey. *arXiv preprint arXiv:2312.10997* (2023).
- [7] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. Yih, T. Rocktäschel, S. Riedel, and D. Kiela. 2020. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In *Advances in Neural Information Processing Systems*, Vol. 33. 9459-9474.
- [8] S. Raja, A. Mondal, and C. V. Jawahar. 2022. Visual Understanding of Complex Table Structures from Document Images. In *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision*. 2543-2552.
- [9] M. Krichen. 2023. Formal Methods and Validation Techniques for Ensuring Automotive Systems Security. *Electronics* 14, 3 (2023), 666. <https://doi.org/10.3390/electronics14030666>
- [10] H. Liu, M. Galindo, H. Xie, L. Wong, H. Shuai, Y. Li, and W. Cheng. 2024. Lightweight Deep Learning for Resource-Constrained Environments: A Survey. *arXiv preprint arXiv:2404.07236* (2024).
- [11] V. D. Viellieber and M. Aßenmacher. 2020. Pre-trained language models as knowledge bases for Automotive Complaint Analysis. *arXiv preprint arXiv:2012.02558* (2020).
- [12] M. Ghaleb, H. Zolfaghariania, and S. Taghipour. 2020. Real-time production scheduling in the Industry-4.0 context: Addressing uncertainties in job arrivals and machine breakdowns. *Computers & Industrial Engineering* 123 (2020), 105031. <https://doi.org/10.1016/j.cie.2020.105031>
- [13] J. B. Gruber and M. Weber. 2024. rollama: An R package for using generative large language models through Ollama. *arXiv preprint arXiv:2404.07654* (2024).
- [14] R. Hussain and S. Zeadally. 2019. Autonomous Cars: Research Results, Issues and Future Challenges. *IEEE Communications Surveys & Tutorials* 21, 2 (2019), 1275-1313. <https://doi.org/10.1109/COMST.2018.2869360>
- [15] Q. Ai, T. Bai, Z. Cao, Y. Chang, J. Chen, Z. Chen, Z. Cheng, S. Dong, Z. Dou, F. Feng, S. Gao, J. Guo, X. He, Y. Lan, C. Li, Y. Liu, Z. Lyu, W. Ma, J. Ma, and Z. Ren. 2023. Information Retrieval meets Large Language Models: A strategic report from Chinese IR community. *Fundamental Research* 4 (2023), 80-90. <https://doi.org/10.1016/j.fmre.2023.06.009>
- [16] D. Amalfitano, V. De Simone, R. R. Maietta, S. Scala, and A. R. Fasolino. 2019. Using tool integration for improving traceability management testing processes: An automotive industrial experience. *Journal of Systems and Software* 151 (2019), e2171. <https://doi.org/10.1002/smr.2171>
- [17] S. Xiao, Z. Liu, P. Zhang, and N. Muennighoff. 2023. C-Pack: Packaged Resources To Advance General Chinese Embedding. *arXiv preprint arXiv:2309.07597* (2023). <https://doi.org/10.48550/arXiv.2309.07597>
- [18] V. Karpukhin, B. Oguz, S. Min, L. Wu, S. Edunov, D. Chen, and W. Yih. 2020. Dense Passage Retrieval for Open-Domain Question Answering. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. 6769-6781. <https://doi.org/10.18653/v1/2020.emnlp-main.550>
- [19] A. Asai, Z. Wu, Y. Wang, A. Sil, and H. Hajishirzi. 2023. Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection. *arXiv preprint arXiv:2310.11511* (2023). <https://doi.org/10.48550/arXiv.2310.11511>
- [20] W. Chen, Z. Li, and M. Ma. 2024. Octopus: On-device language model for function calling of software APIs. *arXiv preprint arXiv:2404.01549* (2024). <https://doi.org/10.48550/arXiv.2404.01549>
- [21] R. Anantha, S. Vakulenko, Z. Tu, S. Longpre, S. Pulman, and S. Chappidi. 2021. Open-Domain Question Answering Goes Conversational via Question Rewriting. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*. <https://doi.org/10.18653/v1/2021.naacl-main.44>
- [22] S. Reddy, D. Chen, and C.D. Manning. 2019. CoQA: A Conversational Question Answering Challenge. *Transactions of the Association of Computational Linguistics* 7 (2019), 249-266. https://doi.org/10.1162/tacl_a_00266